

# FinancesApp API

## Frontend Development Guide

API Reference for Building the FinancesApp Frontend

Version 1.1 | Generated June 2026

---

### Architecture Overview

Frontend (SPA) → AWS Gateway (API Gateway + Lambda authorizer, exchanges the opaque session reference for a phantom JWT) → FinancesApp\_Api (Controllers) → Modules: User / Credentials / Account CQRS Dispatcher + Event Store → Outbox Processor (BackgroundService) → Projections (read tables) SQL Server (events + reads) | AWS S3 (profile images) | DynamoDB (image metadata + session references) | Redis (cache). Identity.Api is a separate microservice that mints session references (/session) and exchanges them for JWTs (/exchange).

---

# What's new in v1.1

v1.1 is fully backward-compatible with v1.0. Additions:

| Endpoint                                        | Description                                                                                      |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------|
| GET /api/v1.1/accounts/transactions             | Cross-account transaction history. Optional from/to (ISO 8601). Sorted newest-first.             |
| GET /api/v1.1/user/get/{userId}                 | Returns { user, profileImageUrl } — a 1-minute pre-signed S3 GET URL, or null if no image.       |
| POST /api/v1.1/credentials/verify-2fa (updated) | Now uses the phantom-token model: X-Access-Token (HttpOnly, SameSite=Strict) holds an OPAQ       |
| POST /api/v1.1/credentials/logout (new)         | Authenticated via X-Access-Token cookie. Invalidates any active TOTP for the user and clears X-A |

Versioning rule: minor bumps 1.0 → 1.1 → ... → 1.9, then rolls to next major (2.0).

---

## 1. What this API is about

FinancesApp is a personal-finance management API. Authentication uses the **phantom-token pattern**: after 2FA the client holds an **opaque session reference** in an HttpOnly cookie — never a JWT. A gateway exchanges that reference for a short-lived JWT before the request reaches the backend. The pre-2FA step still uses a partial Bearer token. .NET 8 modular monolith — three domains:

| Domain      | Responsibility                                                                                    |
|-------------|---------------------------------------------------------------------------------------------------|
| User        | Profile data: name, email, date of birth, profile image (stored in S3).                           |
| Credentials | Login + password (BCrypt) and TOTP-based two-step authentication.                                 |
| Account     | Financial accounts (Cash, Checking, Credit Card) with balance, debt, deposits, withdrawals, payme |

**Eventual consistency:** every state change is an immutable event. Read endpoints serve projections updated asynchronously (~500ms). Re-fetch after writes where freshness matters.

---

## 2. Base URL and API versioning

| Method      | Example                                    |
|-------------|--------------------------------------------|
| URL segment | /api/v1/... or /api/v1.1/... (recommended) |
| Media type  | Accept: application/json;api-version=1.1   |

Default version is **v1.1**. CORS allows any localhost/127.0.0.1 origin for development. HTTPS redirection is skipped in Development.

### 3. Authentication (two-step TOTP + phantom token)

Login is a two-step flow. Step 1 verifies the password → **partial JWT** (Bearer) + QR code. Step 2 verifies the TOTP code → an **opaque session reference** delivered as an **HttpOnly** cookie + profile image URL. **The client never receives or handles a JWT.**

#### Phantom-token model — what the frontend needs to know

- After 2FA, the X-Access-Token cookie holds an **opaque random reference** — not a JWT. It carries no readable claims; treat it as a meaningless handle.
- An **AWS gateway** (API Gateway + Lambda authorizer) sits in front of the backend. On each call it takes the reference, calls Identity.Api /exchange to get a short-lived JWT (the "phantom token"), and forwards that JWT to FinancesApp\_Api. The frontend sees none of this.
- Only the SHA-256 **hash** of the reference is stored server-side (in DynamoDB), like a password. The raw reference lives only in the cookie.
- For the frontend the mechanics are unchanged from a cookie-session: send requests with `credentials: 'include'` and the browser attaches the cookie automatically. You never parse, decode, or store the token yourself.

*The API Gateway + Lambda authorizer is being rolled out. The reference lifetime is 30 minutes; when it expires or is revoked (logout), calls return 401 — route the user back to /login.*

#### Step 1 — POST /api/v1.1/credentials/login

Send { login, password }. Returns { token: <partialJwt>, qrCodeImage: "data:image/png;base64,..." }. Show the QR code so the user can scan it with an authenticator app.

#### Step 2 — POST /api/v1.1/credentials/verify-2fa

Send { totpCode } with Authorization: Bearer <partialToken>. Returns **only { profileImageUrl }** in the response body. The session reference and username are delivered as **HttpOnly cookies** set by the server:

| Cookie         | Value                                | Flags                             |
|----------------|--------------------------------------|-----------------------------------|
| X-Access-Token | Opaque session reference (NOT a JWT) | HttpOnly, Secure, SameSite=Strict |
| X-Username     | User display name                    | HttpOnly, Secure, SameSite=Strict |

Cookies are sent automatically by the browser on all subsequent same-origin requests. For cross-origin, add `credentials: 'include'` to your fetch options.

```
// verify-2fa 200 response body
{
  "profileImageUrl": "https://s3.amazonaws.com/...?X-Amz-..." // null if no image uploaded
}
```

**profileImageUrl** is a pre-signed S3 GET URL valid for **1 minute**. Use it immediately to display the avatar right after login. After 1 minute, call GET /user/get/{userId} for a fresh URL.

#### Phantom token claims (server-side only — never seen by the client)

For reference: the JWT the gateway mints from the session reference carries the claims below. The frontend never receives this token, so these are documented for backend/gateway context only.

| Claim  | Value                   |
|--------|-------------------------|
| Issuer | https://FinancesApp.com |

| Claim                                 | Value                                                                 |
|---------------------------------------|-----------------------------------------------------------------------|
| Audience                              | https://FinancesAppCustomers.com                                      |
| Algorithm                             | RS256                                                                 |
| token_type                            | "partial" (login Bearer) or "full" (phantom token from /exchange)     |
| userid_enc / sub_enc                  | AES-encrypted user GUID / login (server-side use only)                |
| role / 2fa_verified / account_ids_enc | Present on the full phantom token after successful TOTP verification. |

*Note: profile image URL is neither a claim nor part of the reference. It is returned as a separate top-level field in the verify-2fa response body, valid for 1 minute.*

## Important rules

- TOTP codes are **one-time**: invalidated after successful verify. New login → new QR code (re-pair each session).
- TOTP secrets expire after ~5 minutes. If too slow, restart from /login.
- The x-api-key header is an additional filter on selected endpoints. Not a replacement for the session reference.
- The X-Access-Token cookie is an **opaque reference**, not a JWT — there is nothing to decode. JavaScript cannot read it (HttpOnly) and must not try to.
- The reference is valid for **30 minutes**. On 401, send the user to /login to get a new one.

---

## 4. Rate limiting

Sliding-window limits enforced server-side. Exceeding returns **429 Too Many Requests** with Retry-After header.

| Policy      | Limit                       | Partition key     | Where it applies             |
|-------------|-----------------------------|-------------------|------------------------------|
| global      | 100 req/min + 10 concurrent | IP / user         | every endpoint               |
| auth        | 10 req / 30s                | IP                | POST /credentials/login      |
| verify-totp | 5 req / 30s                 | userId (from JWT) | POST /credentials/verify-2fa |
| delta       | 30 req / 30s                | userId (from JWT) | POST /accounts/delta         |

**Tip:** on 429 read Retry-After and disable the button. For verify-2fa, surface a clear message — users retype codes quickly.

---

## 5. Shared types and conventions

### Money (value object)

```
{  
  "amount": 1500.50,  
  "currency": "BRL"  
}
```

- currency: 3-letter ISO 4217 code (BRL, USD, EUR, ...).
- amount: rounded to 2 decimals (MidpointRounding.AwayFromZero).
- All arithmetic is same-currency; no auto-conversion.

### Enumerations

| Enum                   | Values                                                                   |
|------------------------|--------------------------------------------------------------------------|
| AccountType            | 0 = Cash, 1 = Checking, 2 = CreditCard                                   |
| AccountStatus          | 0 = Active, 1 = Closed                                                   |
| OperationType          | 0 = MoneyTransaction (deposit/withdraw), 1 = Payment, 2 = CreditPurchase |
| TransactionKind (v1.1) | 0 = Deposit, 1 = Withdraw, 2 = CreditCardPayment, 3 = CreditCardChange   |

### General conventions

- IDs are GUIDs (UUID v4 strings).
- Dates use ISO 8601 with offset, e.g. 2026-04-08T12:30:00+00:00.
- 400 errors return a plain string body. 404 returns no body.
- List endpoints return [] (not 404) when empty.
- Write responses are eventually consistent: re-read after ~1s.

---

## 6. User endpoints

**Base route:** /api/v1.1/user **Auth:** not required.

### GET /api/v1.1/user

Get all users

|        |                                |
|--------|--------------------------------|
| Auth   | None                           |
| Status | Body / Notes                   |
| 200    | Array of User objects          |
| 200    | Returns [] when no users exist |

### User object schema

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "John Doe",
    "email": "john@example.com",
    "registeredAt": "2026-01-15T00:00:00+00:00",
    "modifiedAt": "2026-01-15T00:00:00+00:00",
    "dateOfBirth": "1990-05-20T00:00:00+00:00",
    "profileImage": "profile-images/{userId}/profile",
    "isDeleted": false,
    "age": 35
  }
]
```

### GET /api/v1.1/user/get/{userId}

Get a single user by id — includes a fresh pre-signed profile image URL

|        |                                                                                                |
|--------|------------------------------------------------------------------------------------------------|
| Auth   | None                                                                                           |
| userId | GUID string                                                                                    |
| Status | Body / Notes                                                                                   |
| 200    | { "user": { ...User object... }, "profileImageUrl": "https://s3.amazonaws.com/...?X-Amz-..." } |
| 200    | profileImageUrl is null if the user has no profile image                                       |
| 400    | "Invalid Id"                                                                                   |
| 404    | User not found                                                                                 |

- profileImageUrl is valid for 1 minute. Use it immediately; do not cache across sessions.

### POST /api/v1.1/user/create

Create a new user

|      |      |
|------|------|
| Auth | None |
|------|------|

### Request body

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "dateOfBirth": "1990-05-20T00:00:00+00:00",
}
```

```
    "profileImage": ""
  }
```

| Status | Body / Notes                                                          |
|--------|-----------------------------------------------------------------------|
| 200    | GUID of the created user                                              |
| 400    | Validation failed (name <=100, email contains @ and <=50, age 16-120) |

### PUT /api/v1.1/user/update/{userId}

Update user — name/email are optional; omit or send empty to keep existing values

|      |      |
|------|------|
| Auth | None |
|------|------|

#### Request body

```
{
  "id":          "550e8400-...",
  "name":        "John Updated",
  "email":       "john.new@example.com",
  "dateOfBirth": "1990-05-20T00:00:00+00:00",
  "profileImage": "<base64-encoded image or empty string>"
}
```

| Status | Body / Notes             |
|--------|--------------------------|
| 200    | GUID of the updated user |
| 400    | Update failed            |

*Profile image upload: base64-encoded JPEG/PNG/WebP, max 2 MB. Processed asynchronously — updated URL appears in GET /user/get/{userId} after ~500ms+. Send empty string to leave existing image unchanged.*



---

## 7. Credentials endpoints

**Base route:** /api/v1.1/credentials **Auth:** mostly none; verify-2fa needs a partial token.

### GET /api/v1.1/credentials/user/{userId}

Look up credentials by user id (no password returned)

|        |                        |
|--------|------------------------|
| Auth   | None                   |
| userId | GUID string            |
| Status | Body / Notes           |
| 200    | UserCredentials object |
| 400    | "Invalid Id"           |
| 404    | Not found              |

### GET /api/v1.1/credentials/login/{login}

Look up credentials by email (no password returned)

|        |                         |
|--------|-------------------------|
| Auth   | None                    |
| login  | email string, non-empty |
| Status | Body / Notes            |
| 200    | UserCredentials object  |
| 400    | "Login cannot be empty" |
| 404    | Not found               |

### POST /api/v1.1/credentials/login

Step 1: verify password, generate TOTP secret, return partial JWT + QR code

|            |                     |
|------------|---------------------|
| Auth       | None                |
| Rate limit | 10 req / 30s per IP |

#### Request body

```
{
  "login": "john@example.com",
  "plainPassword": "SecurePass1!"
}
```

| Status | Body / Notes                                                            |
|--------|-------------------------------------------------------------------------|
| 200    | { "token": "<partialJwt>", "qrCodeImage": "data:image/png;base64,..." } |
| 401    | Invalid credentials                                                     |
| 500    | TOTP secret could not be persisted                                      |

## POST /api/v1.1/credentials/verify-2fa

Step 2: verify 6-digit TOTP, set opaque session reference as HttpOnly cookie, return profile image URL

|            |                                      |
|------------|--------------------------------------|
| Auth       | Authorization: Bearer <partialToken> |
| Rate limit | 5 req / 30s per user                 |

### Request body

```
{
  "totpCode": "123456"
}
```

### 200 Response body

```
{
  "profileImageUrl": "https://s3.amazonaws.com/bucket/profile-images/{userId}/profile?X-Amz-Algorithm=..."
  // null if the user has no profile image uploaded
}
```

### 200 Cookies set

| Cookie         | Content                  | Flags                                     |
|----------------|--------------------------|-------------------------------------------|
| X-Access-Token | Opaque session reference | HttpOnly, Secure, SameSite=Strict, 30 min |
| X-Username     | User display name        | HttpOnly, Secure, SameSite=Strict         |

*profileImageUrl is a direct S3 HTTPS URL — use it as an image src with no extra headers. It expires in 1 minute. After expiry, call GET /user/get/{userId} for a fresh one.*

| Status | Body / Notes                                                       |
|--------|--------------------------------------------------------------------|
| 200    | { "profileImageUrl": "..." } + X-Access-Token + X-Username cookies |
| 400    | Invalid code format (must be 6 digits)                             |
| 401    | Not partial / TOTP expired / wrong code / no active TOTP           |
| 404    | User credentials not found                                         |

## POST /api/v1.1/credentials/logout NEW in 1.1

Sign the user out. Identifies the user from the X-Access-Token HttpOnly cookie (the gateway resolves the session reference to a JWT before the backend sees it), invalidates any active TOTP for the user (server-side, event-sourced), and clears both auth cookies.

|        |                                                                          |
|--------|--------------------------------------------------------------------------|
| Auth   | Cookie: X-Access-Token (session reference)                               |
| Body   | None                                                                     |
| Status | Body / Notes                                                             |
| 200    | Empty body. X-Access-Token and X-Username cookies are cleared (expired). |
| 401    | Token missing or missing user identity claim.                            |
| 400    | Invalid UserId in token.                                                 |

*UserId is read from the encrypted userid\_enc claim — no body needed. After the call returns 200, the browser drops the auth cookies automatically; redirect to /login.*

```
// Logout – no body, cookie carries the token
await fetch(`${BASE_URL}/api/v1.1/credentials/logout`, {
  method: "POST",
  credentials: "include", // sends X-Access-Token
});
```

```
// Cookies are cleared – route the user to /login  
redirectToLogin();
```

## POST /api/v1.1/credentials

Create credentials for an existing user (lookup by email)

|      |      |
|------|------|
| Auth | None |
|------|------|

### Request body

```
{  
  "email":      "john@example.com",  
  "plainPassword": "SecurePass1!"  
}
```

| Status | Body / Notes                                    |
|--------|-------------------------------------------------|
| 200    | GUID of the created credentials                 |
| 400    | "Failed to create credentials"                  |
| 404    | No user with that email — create the user first |

*Password: min 8 chars, stored as BCrypt hash. Never echoed back.*

## PUT /api/v1.1/credentials/{userId}

Update password

|      |      |
|------|------|
| Auth | None |
|------|------|

### Request body

```
{
  "userId": "550e8400-...",
  "newPlainPassword": "NewSecurePass1!"
}
```

| Status | Body / Notes                                   |
|--------|------------------------------------------------|
| 200    | "Credentials updated successfully"             |
| 400    | "Invalid Id" or "Failed to update credentials" |

## DELETE /api/v1.1/credentials/{userId}

Delete credentials

|        |                                                |
|--------|------------------------------------------------|
| Auth   | None                                           |
| userId | GUID string                                    |
| Status | Body / Notes                                   |
| 200    | "Credentials deleted successfully"             |
| 400    | "Invalid Id" or "Failed to delete credentials" |

## POST /api/v1.1/credentials/rebuild-projection/{userId}

Rebuild credentials read model from event store (admin / recovery only)

|        |                                      |
|--------|--------------------------------------|
| Auth   | None (ops use)                       |
| userId | GUID string                          |
| Status | Body / Notes                         |
| 200    | "Projection rebuilt for user <guid>" |
| 400    | Rebuild failed                       |

---

## 8. Account endpoints

**Base route:** `/api/v1.1/accounts` **Auth:** authenticated session required — the X-Access-Token cookie (session reference) is sent automatically and the gateway turns it into a full phantom token. `GetAccounts` and `GetTransactionHistory` require the *full* (post-2FA) token.

### Account object schema

```
{
  "id": "550e8400-...",
  "userId": "660e8400-...",
  "balance": { "amount": 1500.50, "currency": "BRL" },
  "creditLimit": { "amount": 3000.00, "currency": "BRL" },
  "currentDebt": { "amount": 0.00, "currency": "BRL" },
  "paymentDate": null, "dueDate": null, "payedAt": null,
  "status": 0, // 0=Active, 1=Closed
  "type": 0, // 0=Cash, 1=Checking, 2=CreditCard
  "createdAt": "2026-01-15T00:00:00+00:00",
  "closedAt": null,
  "updatedAt": "2026-04-08T12:00:00+00:00"
}
```

*creditLimit/currentDebt/paymentDate/dueDate/payedAt only meaningful on CreditCard (type=2).*

### GET `/api/v1.1/accounts`

List the caller's accounts

|        |                                   |
|--------|-----------------------------------|
| Auth   | Cookie: X-Access-Token (full JWT) |
| Status | Body / Notes                      |
| 200    | Array of Account objects          |
| 401    | Token missing / not a full token  |

*UserId is taken from the encrypted `userid_enc` claim — do not pass it in the body or query string.*

### GET `/api/v1.1/accounts/{accountId}`

Get a single account by id

|           |                        |
|-----------|------------------------|
| Auth      | Cookie: X-Access-Token |
| accountId | Account GUID           |
| Status    | Body / Notes           |
| 200       | Account object         |
| 400       | "Invalid Id"           |
| 404       | Account not found      |

### GET `/api/v1.1/accounts/active`

Active accounts only

|        |                                            |
|--------|--------------------------------------------|
| Auth   | Cookie: X-Access-Token                     |
| Status | Body / Notes                               |
| 200    | Array of Account objects (status = Active) |

## POST /api/v1.1/accounts

Create a new account

|      |                        |
|------|------------------------|
| Auth | Cookie: X-Access-Token |
|------|------------------------|

### Request body

```
{
  "userId": "660e8400-...",
  "balance": { "amount": 1000.00, "currency": "BRL" },
  "type": 0
}
```

| Status | Body / Notes                   |
|--------|--------------------------------|
| 200    | "Account created successfully" |
| 400    | "Failed to create account"     |

## POST /api/v1.1/accounts/delta

Apply a financial transaction (deposit, withdraw, payment, purchase)

|            |                        |
|------------|------------------------|
| Auth       | Cookie: X-Access-Token |
| Rate limit | 30 req / 30s per user  |

### Request body

```
{
  "userId": "660e8400-...",
  "accountId": "550e8400-...",
  "amount": 250.75,
  "currency": "BRL",
  "operationType": 0,
  "requestedAt": "2026-04-08T12:30:00+00:00"
}
```

| Status | Body / Notes                                                                        |
|--------|-------------------------------------------------------------------------------------|
| 200    | "Delta applied successfully"                                                        |
| 400    | Business-rule failure (insufficient funds, currency mismatch, closed account, etc.) |
| 404    | Account not found                                                                   |

MoneyTransaction(0): positive=deposit, negative=withdrawal (Cash/Checking). Payment(1): reduces credit card debt. CreditPurchase(2): increases debt. requestedAt optional — server stamps UTC now when omitted.

---

## 8.1 Transaction history (new in v1.1)

Unified time-ordered feed across all of the authenticated user's accounts.

### GET /api/v1.1/accounts/transactions NEW in 1.1

|        |                                                       |
|--------|-------------------------------------------------------|
| Since  | v1.1                                                  |
| Auth   | Cookie: X-Access-Token (full JWT)                     |
| from   | Optional. ISO 8601 datetime. Lower bound (inclusive). |
| to     | Optional. ISO 8601 datetime. Upper bound (inclusive). |
| Status | Body / Notes                                          |
| 200    | Array of AccountTransaction objects, newest-first     |
| 400    | "from" must be earlier than or equal to "to".         |
| 401    | Token missing / not a full token                      |

### AccountTransaction schema

```
[
  {
    "eventId": "11111111-...",
    "accountId": "550e8400-...",
    "timestamp": "2026-04-08T12:30:00+00:00",
    "kind": 0,
    "amount": { "amount": 250.75, "currency": "BRL" }
  }
]
```

| Kind                  | Meaning                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------|
| 0 - Deposit           | Money in to Cash/Checking. amount is positive.                                                                     |
| 1 - Withdraw          | Money out of Cash/Checking. amount is positive (magnitude).                                                        |
| 2 - CreditCardPayment | Statement payment on a Credit Card (raised via PayCreditCardDebt).                                                 |
| 3 - CreditCardChange  | Debt change from ApplyDelta on a credit card. amount = NewDebt - CurrentDebt: positive=purchase, negative=payment. |

Reflects the event store in real time — newer than projection-based Account read endpoints. O(accounts x events); query narrower windows for long-lived users.

---

## 9. Health check

### GET /health

Service liveness + infrastructure probes

|        |                                              |
|--------|----------------------------------------------|
| Auth   | None                                         |
| Status | Body / Notes                                 |
| 200    | JSON describing each registered health check |

```
{
  "status": "Healthy",
  "duration": "00:00:00.123",
  "checks": [
    { "name": "SQL Database Check", "status": "Healthy", "tags": ["database","critical"] },
    { "name": "S3 Bucket Check", "status": "Healthy", "tags": ["aws","storage"] },
    { "name": "DynamoDB Check", "status": "Healthy", "tags": ["aws","storage"] }
  ]
}
```

*All three checks run on every /health poll. S3 uses `GetBucketLocation`; DynamoDB uses `DescribeTable` (returns `Degraded` if table is not `ACTIVE`).*



---

## 10. Profile image guide

Profile images are stored in AWS S3. The frontend never holds credentials — all access is via short-lived pre-signed URLs.

### Upload a profile image

Send the image as base64 in `profileImageUrl` field of PUT `/api/v1.1/user/update/{userId}`. Accepted: JPEG, PNG, WebP, max 2 MB. Upload is async — updated URL appears in GET `/user/get/{userId}` after ~500ms+.

### Get the pre-signed URL

Two ways to get the URL — both return the same S3 pre-signed URL, valid for 1 minute:

| Source                                                                   | When to use                                                                                               |
|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| POST <code>/credentials/verify-2fa</code> → <code>profileImageUrl</code> | Immediately after login. Use the URL from the response body to display the avatar right on the post-login |
| GET <code>/user/get/{userId}</code> → <code>profileImageUrl</code>       | Any time after login, or after the 1-minute window expires. Always returns a fresh URL.                   |

### Display a profile image

```
// After verify-2fa – response body contains only profileImageUrl:
const { profileImageUrl } = await verifyTotpResponse.json();
if (profileImageUrl) imgElement.src = profileImageUrl;

// After expiry – refresh:
const { user, profileImageUrl: freshUrl } =
  await (await api(`/api/v1.1/user/get/${userId}`)).json();
if (freshUrl) imgElement.src = freshUrl;
```

The URL is a plain HTTPS GET — no extra headers required.

**Important:** pre-signed URLs expire in **1 minute**. Do not cache across sessions or store in persistent state. Each upload overwrites the previous image (fixed key: `profile-images/{userId}/profile`).

---

## 11. Frontend implementation guidance

### Recommended user flow

- Sign-up: POST /user/create, then POST /credentials with the same email.
- Login step 1: POST /credentials/login — store partial token in memory, render QR.
- Login step 2: POST /credentials/verify-2fa — opaque session reference set as HttpOnly cookie automatically. Use **profileImageUrl** from the response body to display the avatar immediately.
- Dashboard: GET /accounts for balances, GET /accounts/transactions for activity feed.
- Account detail: GET /accounts/{accountId}.
- Transactions: POST /accounts/delta.
- Profile / avatar: GET /user/get/{userId} for fresh URL + user data. PUT /user/update/{userId} to update.
- Logout: POST /credentials/logout with credentials: 'include' — server invalidates the active TOTP and clears X-Access-Token + X-Username cookies. Redirect to /login afterwards.

### Error handling cheatsheet

| Status | Meaning                                                    |
|--------|------------------------------------------------------------|
| 400    | Plain-string error in body. Show it to the user.           |
| 401    | Token missing/expired or wrong type. Send user to /login.  |
| 404    | Empty body. Render not-found state.                        |
| 429    | Rate limited. Read Retry-After header, disable the action. |
| 500    | Server error. Offer retry, log to monitoring.              |

### Things to keep in mind

- No DELETE-user endpoint; only credentials can be deleted.
- Money is always { amount, currency }. Never send a bare number.
- Credit card fields are backend-computed. Display only, do not edit directly.
- Read endpoints eventually consistent (~500ms). Re-fetch after writes.
- TOTP regenerates every login — user re-pairs each session.
- Auth is **phantom-token**: the client holds an opaque session reference, never a JWT. A gateway exchanges the reference for a short-lived JWT before the backend sees the request.
- The session reference is delivered as an **HttpOnly cookie (X-Access-Token)** — JavaScript cannot read it and there is nothing to decode. The browser sends it automatically on every request. Do **not** try to store, parse, or manage any token in JavaScript.
- For cross-origin requests, add credentials: 'include' to your fetch/XHR options.
- profileImageUrl expires in 1 min. Re-call GET /user/get/{userId} to refresh.

## Suggested HTTP client wrapper

```
// pseudo-code, framework-agnostic
const API_VERSION = "1.1";

async function api(path, init = {}) {
  const headers = new Headers(init.headers);
  headers.set("Accept", `application/json;api-version=${API_VERSION}`);
  // For the partial-auth step, set the Authorization header explicitly:
  // headers.set("Authorization", `Bearer ${partialToken}`);
  // After verify-2fa, the opaque session reference travels as the X-Access-Token cookie – no header needed.
  if (init.body && !headers.has("Content-Type"))
    headers.set("Content-Type", "application/json");

  const res = await fetch(`${BASE_URL}${path}`, {
    ...init,
    headers,
    credentials: "include", // sends cookies cross-origin
  });
  if (res.status === 401) redirectToLogin();
  if (res.status === 429) {
    const retry = Number(res.headers.get("Retry-After") ?? 1);
    throw new RateLimited(retry);
  }
  return res;
}

// Step 1: login – get partial token and QR code
const { token: partialToken, qrCodeImage } =
  await (await fetch(`${BASE_URL}/api/v1.1/credentials/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ login, plainPassword }),
    credentials: "include",
  })).json();
renderQrCode(qrCodeImage);

// Step 2: verify-2fa – opaque session reference set as cookie automatically
const { profileImageUrl } =
  await (await fetch(`${BASE_URL}/api/v1.1/credentials/verify-2fa`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${partialToken}`,
    },
    body: JSON.stringify({ totpCode }),
    credentials: "include",
  })).json();
// X-Access-Token cookie (opaque reference) is now set – no token to store in JS
if (profileImageUrl) showAvatar(profileImageUrl);

// All subsequent requests – cookie sent automatically:
const accounts = await (await api("/api/v1.1/accounts")).json();
const txns = await (await api(
  `/api/v1.1/accounts/transactions?from=${encodeURIComponent(from)}`
)).json();

// Refresh profile image URL after 1 minute
const { user, profileImageUrl: freshUrl } =
  await (await api(`/api/v1.1/user/get/${userId}`)).json();
```

```
if (freshUrl) showAvatar(freshUrl);
```